Lane Keeping Assist

Video Analysis

Optional Homework · Introduction to Vehicle and Sensors

Submission guide: what I handed in, how to run it, how it works, and where it fails



Student: _____

Matriculation no.: _____

October 2026

Code: https://github.com/myamin66/Lane-Keeping-Assist-Video-Analysis

1. What I submitted

The assignment asked for five things. This table shows where each one is in the submission folder.

Asked for	Where to find it
Source code	01_Source_Code/ (also on GitHub, see the title page)
Brief instructions for running it	Section 3 of this guide, and 01_Source_Code/README.md
Short annotated video where it works well	02_Annotated_Videos/01_good_project_concrete_and_shadows.mp4
Challenging case / limitations	02_Annotated_Videos/02_ok_challenge_bridge_shadow_and_seams.mp4 (hard but handled) 02_Annotated_Videos/03_limit_harder_glare.mp4 (uncertain detections) 02_Annotated_Videos/04_limit_harder_hairpin_wrong_marking.mp4 (wrong detection)
Visual walkthrough of the pipeline	Section 4 of this guide, the screenshots in 03_Pipeline_Walkthrough/screenshots/, and the diagnostic videos in 03_Pipeline_Walkthrough/diagnostic_videos/
ODD assessment (for the discussion)	Section 6 of this guide

Folder layout

```
Submission/
├── 00_Submission_Guide.pdf / .docx      this document
├── 01_Source_Code/
│   ├── run_lka.py                       main script (video in -> annotated video out)
│   ├── save_pipeline_steps.py           saves one picture per pipeline stage
│   ├── calibrate_camera.py              one-time camera calibration
│   ├── settings.py                      every tuned number in one place
│   ├── image_processing.py              undistort, masks, ROI, bird's-eye warp
│   ├── lane_search.py                   sliding windows + curve fit
│   ├── lane_tracker.py                  score, checks, smoothing, holding
│   ├── overlay.py                       everything that is drawn
│   ├── pipeline.py                      glues the steps together
│   ├── camera_cal/  camera_calibration.npz
│   ├── videos/                          put the input videos here
│   ├── requirements.txt
│   └── README.md                        detailed technical explainer
├── 02_Annotated_Videos/                 4 short clips (good, challenging, 2x limits)
└── 03_Pipeline_Walkthrough/
    ├── diagnostic_videos/               3 clips showing all stages side by side
    └── screenshots/                     stage pictures + result frames
```

2. The project in short

The program takes a forward-facing road video and, on every frame, finds the left and right boundary of the lane the car is driving in. It draws both boundaries in different colours, marks the lane centre, and shows where the car sits relative to it. Each boundary gets its own confidence score. When a boundary can't be found, the overlay says so instead of quietly showing an old position as if it were new. It's offline video analysis only, with no vehicle control.

- **Method:** classical image processing in Python with OpenCV and NumPy, no neural network
- **Footage:** the three Udacity CarND-Advanced-Lane-Lines videos (project, challenge, harder challenge) and their chessboard calibration photos

3. How to run it, step by step

Step 1: Install Python and the two packages

Install Python 3.9 or newer from python.org. Then open a terminal inside 01_Source_Code and run:

```
pip install -r requirements.txt
```

Step 2: Put the input videos in place

The input videos aren't in the submission, to keep it small. Download project_video.mp4, challenge_video.mp4 and harder_challenge_video.mp4 from the Udacity repository (github.com/udacity/CarND-Advanced-Lane-Lines) or from the videos/ folder of my GitHub repository, and put them in 01_Source_Code/videos/.

Step 3 (optional): Camera calibration

camera_calibration.npz is already included, so this step can be skipped. To redo it:

```
python calibrate_camera.py
```

This finds the chessboard corners in camera_cal/ (17 of the 20 photos are usable; the other three cut off part of the board) and saves the camera matrix and distortion coefficients.

Step 4: Process a video

Always run the commands from inside 01_Source_Code:

```
python run_lka.py videos/project_video.mp4 --diagnostic
python run_lka.py videos/challenge_video.mp4 --camera challenge --diagnostic
python run_lka.py videos/harder_challenge_video.mp4 --diagnostic
```

Option	What it does
--diagnostic	also writes output/<name>_diagnostic.mp4 with six pipeline stages in a grid
--camera challenge	camera profile for challenge_video.mp4, which was filmed with a different camera mount (Section 4, stage 2)
--start 18 --end 30	only process this time range, in seconds
--output file.mp4	choose the output file name (default: output/<name>_lka.mp4)

It runs at about 6 frames per second, so the 50-second project video takes 3 to 4 minutes. At the end the script prints how often each boundary was DETECTED, UNCERTAIN, HOLDING or LOST. That summary makes it quick to compare settings.

Step 5: Save the pipeline pictures for one moment

```
python save_pipeline_steps.py videos/project_video.mp4 --time 22
python save_pipeline_steps.py videos/challenge_video.mp4 --time 5 --camera challenge
```

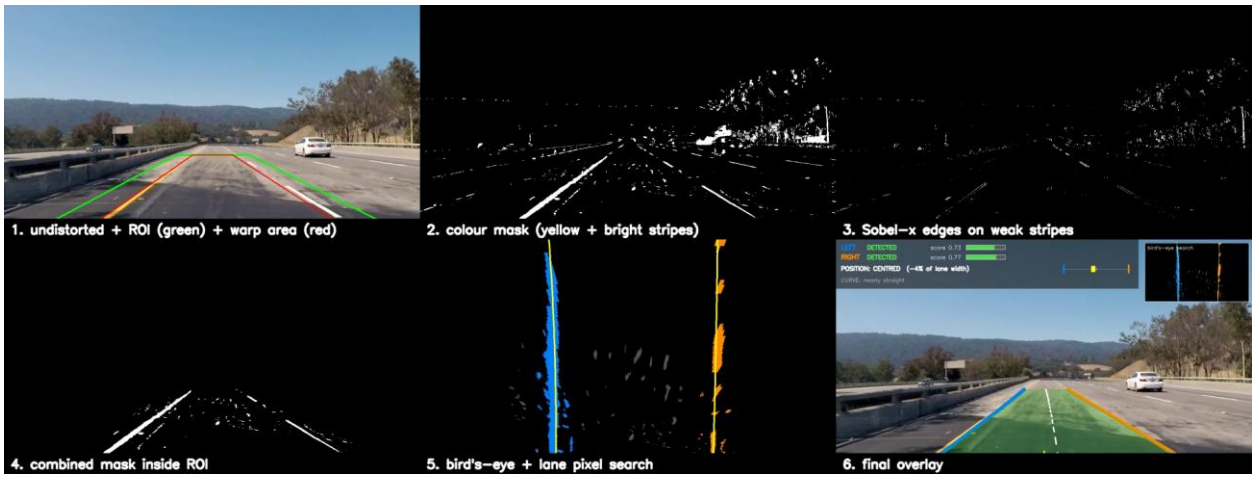
The frames before the chosen time are processed too, so the tracker has its normal history. The pictures go to screenshots/<video>_<time>s/.

If something goes wrong

- **"Could not open video":** check the path, and run from inside 01_Source_Code.
- **Video doesn't play:** OpenCV writes the mp4v codec. Use VLC, or convert with: ffmpeg -i in.mp4 -c:v libx264 -pix_fmt yuv420p out.mp4. The clips in the submission are already H.264.
- **Changing the tuning:** all thresholds and limits are in settings.py, each with a comment.

4. Visual walkthrough of the pipeline

All pictures in this section come from the same frame: project_video.mp4 at 22 s, on a light concrete bridge. The diagnostic videos show the same stages moving.



All stages at once (this is one frame of a diagnostic video)

Stage 1: Undistort the frame

The lens bends straight lines, mostly near the edges. The chessboard photos give the camera matrix and distortion coefficients (cv2.calibrateCamera), and every frame is corrected with cv2.undistort. Without this step a straight road would look slightly curved in the bird's-eye view.



Calibration check: original chessboard photo (left) and undistorted (right)

Stage 2: Region of interest and warp area



Green: region of interest. Red: the four points that get warped to a rectangle.

- **Green trapezoid (ROI).** Everything outside the green trapezoid (sky, trees, other lanes, far side of the road) is ignored. It is wider than the red one, so the lines stay inside when the car drifts or the road bends.
- **Red trapezoid (warp area).** These four road points are mapped onto a rectangle to get the bird's-eye view. I picked them on a straight piece of road so that the lane lines come out vertical.

Why the challenge video needs its own setting: at first the challenge video kept failing my "lines must be parallel" check. When I mapped the detected lines back into the camera image and intersected them, the point where they meet in the distance was about 45 px further right and 13 px lower than in the project video. That means a different camera mount. I added a second set of points (--camera challenge), and the left boundary went from 44 % to 95 % of frames detected.

Stage 3: Colour mask (finding paint)



Colour mask: yellow OR bright stripe



Edge mask: Sobel-x on weak stripes

Yellow paint is an HSV colour range. White paint took three tries, and these are worth explaining in the discussion:

1. **Try 1.** Fixed brightness threshold: fine on dark asphalt, but on the light concrete bridges the whole road passed it.
2. **Try 2.** "Brighter than the local average": fixed the bridges, but it also fired on the sunlit side of concrete barriers and along shadow borders.
3. **Try 3, the stripe test (used now).** A paint pixel has to be brighter than the road 12 px to its left AND 12 px to its right at the same time. Paint is a thin stripe, so it passes. A barrier or shadow border is only brighter on one side, so it fails. On the challenge video this cut the median fit error from about 25 px to about 7 px.

Stage 4: Edge mask

Sobel in the x direction on the lightness channel, because lane lines are close to vertical in the image. On its own it lit up every shadow edge and tar seam, so an edge pixel only counts if it also passes a much weaker stripe test. That keeps some faded paint without bringing the shadows back.

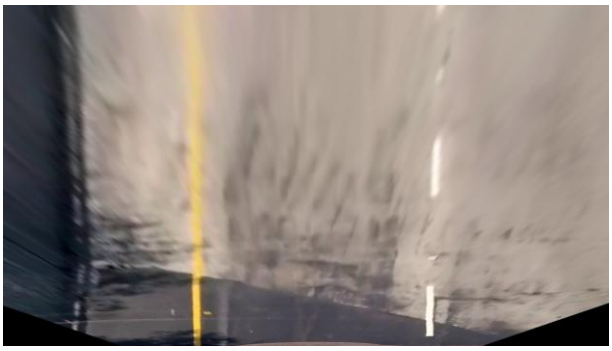
Stage 5: Combine and cut to the ROI

Colour mask OR edge mask, then everything outside the green trapezoid is set to black.



Combined mask inside the ROI

Stage 6: Bird's-eye view



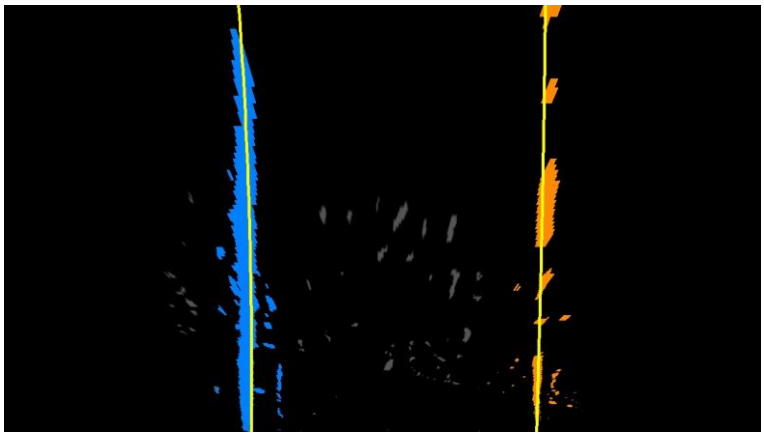
Bird's-eye view of the road



Bird's-eye view of the combined mask

cv2.warpPerspective maps the red trapezoid onto a rectangle. Now the lane looks as if seen from above: both lines are parallel and about 480 px apart.

Stage 7: Search for lane pixels and fit a curve



Left pixels blue, right pixels orange, fitted curves yellow

- **Sliding windows (first frame or after losing a line).** If there's no history, a column histogram of the lower half shows where each line starts. Nine stacked windows then climb up the image and re-centre on the pixels they find. In a gap between dashes they just keep going straight up.
- **Search around the previous curve (normal case).** Otherwise only pixels within ±60 px of last frame's curve are used. This is faster and doesn't jump onto a different marking.

The pixels of each side are fitted with $x = a \cdot y^2 + b \cdot y + c$ (np.polyfit). x is written as a function of y because in the bird's-eye view each row has one x per line.

Stage 8: Decide what to believe (tracking)

Every new fit goes through three steps:

1. Score it. The confidence score is the average of four parts, each 0 to 1: number of supporting pixels, how much of the image height they cover, how well the curve fits them, and how close it is to recent frames. A fit is rejected if the score is below 0.35 or it moved more than 70 px since the last frames.
2. Check both sides together: they must not cross, the lane width at the bottom must be 330 to 640 px, and they must be roughly parallel. If the pair fails, the weaker new side is dropped.
3. Update the state of each boundary and smooth accepted fits over the last 6 frames.

State	When	How it is drawn
DETECTED	fresh fit, score ≥ 0.60	thick solid line
UNCERTAIN	fresh fit, score 0.35–0.60	thin solid line
HOLDING (n)	nothing usable this frame; old curve kept up to 12 frames (~0.5 s)	dashed, greyed line; score fades; reason shown
LOST	held longer than 12 frames	not drawn; next search starts fresh

The score is a heuristic quality measure, not a calibrated probability. I haven't checked it against ground truth, so 0.8 doesn't mean an 80 % chance of being right.

Stage 9: Final overlay



Final overlay

On screen	Meaning
Blue / orange line	left / right lane boundary (style shows its state, see above)
Green / amber lane area	green when both sides are DETECTED, amber if either one is not

Dashed white line	estimated lane centre
Yellow bar + white dot + arrow	car centre, lane centre and the offset between them
LEFT / RIGHT rows	state, score and score bar per boundary, plus the reason when a line was rejected (jumped, not parallel, lane width, ...)
POSITION	CENTRED / SHIFTED LEFT or RIGHT / CLOSE TO LEFT or RIGHT LINE, with the offset in % of lane width; marked "uncertain" if a side is only held, "unknown" if a side is lost
CURVE	rough curve radius and direction (not physically calibrated)
Inset, top right	the bird's-eye search from stage 7

Position = (car centre − lane centre) / lane width at the bottom of the bird's-eye image. The camera sits in the middle of the car, so the car centre is the middle of the image. Below 8 % it counts as centred, from 20 % on as close to the line. A car is about half a lane wide, so at about 25 % its side is already on the line.

5. Results and the annotated videos

Video	Left boundary	Right boundary
project_video	DETECTED 100 %	DETECTED 100 %
challenge_video	DETECTED 95 %, HOLDING 5 %	DETECTED 98 %, HOLDING 2 %
harder_challenge_video	DETECTED 44 %, UNCERTAIN 6 %, HOLDING 33 %, LOST 18 %	DETECTED 54 %, UNCERTAIN 3 %, HOLDING 31 %, LOST 12 %

These numbers say how often the system thinks it has a line, not how often it is right. I checked correctness by watching the videos. Clip 4 below shows a case where it is confidently wrong.

Clip 1: Works well (project video, 18–30 s)



A light concrete bridge followed by tree shadows. Both boundaries stay DETECTED (scores around 0.7 to 0.85), the lines sit on the paint, and the overlay doesn't flicker. The yellow solid line and the white dashed line are both followed; the windows carry on through the gaps between dashes.

Clip 2: Challenging but handled (challenge video, full 16 s)



This video has dark and light asphalt patches, a long crack running along the lane, and a dark bridge underpass. The crack and the colour seams are ignored, because they are darker than the road or are a step rather than a stripe. Under the bridge the left score drops to 0.43: the line turns thin (UNCERTAIN), the lane area turns amber, and a few frames later it's back to DETECTED.

Clip 3: Limitation, glare (harder video, 18–27 s)



The sun shines almost straight into the camera and washes the left side out. The stripe test finds no contrast, so the left boundary goes to HOLDING: dashed and greyed, with a fading score and the reason next to it. The position line turns amber and says "uncertain". This is the behaviour I wanted: the overlay admits it doesn't really know.

Clip 4: Limitation, wrong marking in a hairpin (harder video, 39–48 s)



This is the most interesting failure. The bend is so sharp that the lane lines leave the region of interest. The left boundary fails the lane-width check and becomes LOST, and the position says "unknown", which is correct. But the right boundary has slid onto the yellow centre line and still claims DETECTED with a score of 0.74.

Why: the score only measures how line-like the pixels are (count, coverage, fit, consistency). It can't tell whether a line is the right one. The yellow centre line is a perfectly good line, just the wrong marking. A confident wrong answer like this is worse than an honest LOST. A fix would be to check the marking type (colour, single or double) and to let the region of interest follow the curve.

6. Operational Design Domain (ODD)

Where the solution works reliably and where it doesn't, based on the three videos:

Condition	Result	Why
Clear solid or dashed lines, yellow or white	**Works well**	stripe test + yellow mask; windows bridge the dash gaps
Light concrete / patched road surface	**Works well**	stripe test uses relative brightness, so a bright road doesn't flood the mask
Asphalt seams, cracks, tar lines in the lane	**Works well**	darker than the road, or a one-sided step, so not a stripe
Moderate tree / bridge shadows	**Mostly works**	short dropouts become UNCERTAIN or HOLDING and recover
Gentle curves (radius of several hundred m or more)	**Works well**	2nd-order fit, lines stay inside the ROI
Sharp curves, hairpins	**Fails**	lines leave the fixed ROI; parallel/width checks reject the rest
Glare, fast light changes (leaving a forest)	**Fails, honestly**	paint contrast disappears; overlay goes to HOLDING / LOST
Strong sun-through-trees patterns, weak paint	**Weak**	bright spots between shadows can look like stripes
Ambiguous markings (double yellow, road edge)	**Can be wrong**	the score can't tell the right marking from a wrong one (clip 4)
Different camera mount, hills	**Needs setup**	warp points depend on camera height/pitch; a new profile is needed
Lane changes	**Recovers slowly**	old lane held, new one rejected as "jumped", fresh search after ~0.5 s
Night, rain, wet roads, tunnels, snow, exits, merges	**Not tested**	I expect wet-road reflections to pass the stripe test

How the overlay reacts when the lane is no longer clear

It degrades in steps instead of guessing:

1. solid line (DETECTED)
2. thin line, amber lane area (UNCERTAIN)
3. dashed grey line with a fading score and the reason shown, position marked "uncertain" (HOLDING, at most about 0.5 s)
4. nothing drawn, position "unknown", fresh search (LOST)

A position carried over from earlier frames never looks like a new detection.

7. Notes for the discussion

Live demo (about 2 minutes)

```
python run_lka.py videos/project_video.mp4 --start 18 --end 26 --diagnostic
```

```
python run_lka.py videos/harder_challenge_video.mp4 --start 40 --end 46
python save_pipeline_steps.py videos/project_video.mp4 --time 22
```

Points I want to explain

- why the stripe test beat a brightness threshold and a local-average threshold
- why x is fitted as a function of y in the bird's-eye view
- how the score is built and why it isn't a probability
- the four states and how held lines are kept apart from fresh detections
- why the challenge video needed its own camera profile (vanishing point)
- the hairpin failure: a confident wrong detection, and what would fix it

What I would improve next

- estimate a lost side from the other side plus the last known lane width
- a region of interest that follows the curve, for hairpins
- check the marking type (single white vs double yellow)
- calibrate the score on a few hundred hand-labelled frames
- check the stripe width, to reject wet-road reflections and sun spots